

TEC | Tecnológico de Costa Rica

Instituto Tecnológico de Costa Rica

Tarea #2

Estudiante:

Quiriat Mata Araya – 2023379891

Elder Pérez León - 2023166120

Profesor:

Allan Rodríguez Dávila

Curso:

Compiladores e Interpretes

G60

Semestre I 2026

Contents

Descripción del Problema.....	3
Estructura del proyecto:	3
Librerías usadas	4
Compilación:.....	4
Ejecución:.....	4
Flujo de ejecución:.....	4
Archivos de prueba:.....	4
Diseño del programa.....	5
Arquitectura General	5
Analizador Léxico	5
Análisis Semántico	5
Generación de Código Intermedio.....	5
Clase Resultado	6
Pruebas de funcionalidad.....	6
Prueba 1: Código correcto.....	6
Prueba 2: Errores léxicos	7
Prueba 3: Errores sintácticos	7
Prueba 4: Errores semánticos	8
Análisis de resultados	8
Justificación de toma de decisiones.....	9
Uso de la clase Resultado	9
Etiquetas descriptivas vs numéricas	9
Pila separada para break (pilaBreak)	9
Generación condicional del código intermedio	9
Temporales para cada operando	9
Switch sin etiqueta default explícita	9
Justificación de toma de decisiones: validación semántica y estructuras utilizadas	10

Descripción del Problema

El proyecto consiste en el desarrollo de un compilador completo para un lenguaje de programación personalizado. El compilador abarca las fases de análisis léxico, análisis sintáctico, análisis semántico y generación de código intermedio de tres direcciones.

El lenguaje diseñado soporta los siguientes elementos:

- Tipos de datos primitivos: int, float, bool, char, string
- Arreglos bidimensionales de tipo int y float
- Estructuras de control: if/else, do-while, switch/case
- Funciones con parámetros y retorno de valores
- Operaciones aritméticas, relacionales y lógicas
- Entrada y salida estándar mediante cin y cout

El compilador detecta y reporta errores léxicos, sintácticos y semánticos, y únicamente genera código intermedio cuando el programa analizado no contiene errores de ningún tipo.

Estructura del proyecto:

Proyecto/

lib/

jflex.java

java-cup.jar

java-cup-runtime.jar

src/ app/Main.java

lexer/Lexer.flex

parser/

Parser.cup

TablaSimbolos.java

ErroresSemanticos.java

CodigoIntermedio.java

Resultado.java

EjemplosPruebas/

archivo1.txt

archivo2.txt

Librerías usadas

Librería	Uso
JFlex	Generador de analizadores léxicos a partir de especificaciones .flex
CUP (java cup)	Generador de parsers LALR(1) a partir de gramáticas .cup
Java-cup-runtime	Necesario para ejecutar los parsers generados por CUP

Manual de Usuario

Compilación:

Abra una terminal en la raíz del proyecto (carpeta Proyecto/) y ejecute:

- `javac -cp ".;lib/jflex.jar;lib/java-cup-runtime.jar" src/app/Main.java`

Ejecución:

Una vez compilado, ejecute el siguiente comando desde la raíz del proyecto:

- `java -cp ".;lib/jflex.jar;lib/java-cup-runtime.jar;src/app" Main`

Flujo de ejecución:

Al ejecutar Main.java el programa realiza los siguientes pasos de forma autónoma:

- Elimina archivos Java generados previamente: Lexer.java, MyParser.java, sym.java
- Genera Lexer.java desde src/lexer/Lexer.flex usando JFlex
- Genera MyParser.java y sym.java desde src/parser/Parser.cup usando CUP
- Compila en orden: sym → TablaSimbolos/ErroresSemanticos/CodigoIntermedio → Lexer → MyParser
- Para cada archivo .txt en EjemplosPruebas/ (excepto los que empiecen con tokens_):
 - Limpia la tabla de símbolos, errores semánticos y código intermedio
 - Ejecuta el análisis léxico y genera tokens_<nombre>.txt
 - Ejecuta el análisis sintáctico y semántico
 - Imprime tabla de símbolos, errores y veredicto
 - Si no hay errores, genera Codigo3D_<nombre>.txt con el código intermedio

Archivos de prueba:

Para probar el proyecto coloque archivos con extensión .txt en la carpeta EjemplosPruebas/. El sistema los procesará automáticamente en la siguiente ejecución. Los archivos de salida se guardan en la misma carpeta:

- tokens_<nombre>.txt — lista de tokens reconocidos por el leer
- Codigo3D_<nombre>.txt — código intermedio de tres direcciones (solo si no hay errores)

Diseño del programa

Arquitectura General

El compilador está organizado en fases clásicas, donde cada fase toma la salida de la anterior:

- Análisis Léxico (JFlex) — convierte el código fuente en tokens
- Análisis Sintáctico (CUP) — verifica la estructura gramatical y construye el AST implícito
- Análisis Semántico — verifica tipos, scopes y coherencia del programa
- Generación de Código Intermedio — produce código de tres direcciones

Analizador Léxico

Implementado con JFlex a partir del archivo Lexer.flex. Reconoce los siguientes tipos de tokens:

- Palabras reservadas: int, float, bool, char, string, if, else, do, while, switch, case, default, return, break, cin, cout, true, false, empty
- Identificadores: secuencias de letras, dígitos y guiones bajos
- Literales: enteros, flotantes, exponenciales, fracciones, cadenas, caracteres
- Operadores y delimitadores: todos los símbolos del lenguaje
- Comentarios de línea (//) y de bloque ({- -}): ignorados

Análisis Semántico

Realizado durante el parsing mediante acciones en las producciones CUP. Validaciones implementadas:

- Variables declaradas antes de ser usadas
- Compatibilidad de tipos en asignaciones y operaciones
- Funciones declaradas antes de ser llamadas
- Retorno correcto según el tipo de la función
- Uso de break solo dentro de ciclos o switch
- Detección de código inalcanzable después de return
- Índices de arreglos deben ser de tipo int
- cin solo acepta variables de tipo int o float

Generación de Código Intermedio

El código intermedio generado sigue el formato de tres direcciones. Cada instrucción tiene a lo sumo un operador y tres operandos. Características del código generado:

- Temporales numerados: t1, t2, t3...
- Etiquetas descriptivas: if_else_1, if_end_1, do_start_1, switch_case_1, etc.
- Instrucción var para declaraciones
- Instrucciones param para parámetros de función

- Instrucción call para llamadas a función
- Instrucciones read y print para entrada/salida
- Solo se genera si el programa no tiene errores léxicos, sintácticos ni semánticos

Clase Resultado

Para transportar simultáneamente el valor (temporal o nombre) y el tipo semántico de cada expresión, se creó la clase Resultado con dos campos:

- valor: String - nombre del temporal o literal que representa la expresión en el código intermedio
- tipo: String - tipo semántico de la expresión (int, float, bool, char, string, error)

Su método toString() retorna el valor, lo que permite usarla transparentemente donde se espera un String.

Pruebas de funcionalidad

Prueba 1: Código correcto

```
empty ~ __main__ <| |>
|:
|   int ~ x <- 5 !
|   int ~ y <- 3 !
|   int ~ z <- x + y !
|   cout<|z|> !
:|
```

Resultado esperado:

- Sin errores léxicos, sintácticos ni semánticos
- Código intermedio generado correctamente
- ArchivoCodigo3D_<nombre>.txt creado en EjemplosPruebas/

Código 3D generado:

```

1.  main:
2.  t1 = 5
3.  var int x
4.  x = t1
5.  t2 = 3
6.  var int y
7.  y = t2
8.  t3 = x
9.  t4 = y
10. t5 = t3 + t4
11. var int z
12. z = t5
13. t6 = z
14. print t6
15. main_end:

```

Prueba 2: Errores léxicos

Entrada con caracteres inválidos para el lenguaje:

```

empty ~ __main__ <| |>
|:
|   int ~ x = 5 ;
|:

```

Resultado esperado:

- Errores léxicos reportados: '=' y ';' son caracteres no reconocidos
- No se genera código intermedio

```

Archivo: Ejemplo.txt
Error léxico: carácter inesperado '=' en línea 3, columna 13
Error léxico: carácter inesperado ';' en línea 3, columna 17

[ LÉXICO ]

Errores léxicos encontrados: 2
Error léxico: carácter inesperado '=' en línea 3, columna 13
Error léxico: carácter inesperado ';' en línea 3, columna 17
Error léxico: carácter inesperado '=' en línea 3, columna 13
Error sintáctico en línea 3, columna 15: token inesperado '5'
Error léxico: carácter inesperado ';' en línea 3, columna 17
Error sintáctico irrecoverable en línea -1, columna -1. No se pudo continuar el análisis.

```

Prueba 3: Errores sintácticos

Entrada con estructura gramatical incorrecta:

```
empty ~ __main__ <| |>
|:
|   int ~ x <- !
:|
```

Resultado esperado:

- Error sintáctico: token inesperado '!'
- El parser intenta recuperarse
- No se genera código intermedio

```
Errores sintácticos encontrados: 2
Error sintáctico en línea 4, columna 16: token inesperado '!'
? Recuperado: se omitió sentencia inválida hasta '!'
```

Prueba 4: Errores semánticos

Entrada con error de tipos:

```
empty ~ __main__ <| |>
|:
|   int ~ x <- 5 !
|   x <- true !
:|
```

Resultado esperado:

- Error semántico: no se puede asignar bool a variable int
- No se genera código intermedio

```
[ ERRORES SEMÁNTICOS ]
Error semántico en línea 5, columna 5: no se puede asignar un valor de tipo 'bool' a la variable 'x' de tipo 'int'.
```

Análisis de resultados

A continuación, se presenta un resumen de los resultados obtenidos:

Análisis Léxico	Completo
Análisis Sintáctico	Completo
Análisis Semántico	Completo

Código Intermedio	Completo
Recuperación de errores	Completo

Justificación de toma de decisiones

Uso de la clase Resultado

Las producciones del parser originalmente devolvían solo el tipo semántico como String. Al agregar la generación de código intermedio, se necesitaba también el valor (temporal o nombre de variable) de cada expresión. En lugar de usar el formato 'valor|tipo' (que requiere parsing manual y es frágil) o crear estructuras externas, se optó por una clase simple con dos campos públicos y un toString() que devuelve el valor. Esto mantiene compatibilidad con el código existente y hace el código más legible.

Etiquetas descriptivas vs numéricas

Se optó por etiquetas descriptivas (if_else_1, do_start_1) en lugar de etiquetas genéricas (L1, L2) porque facilitan la lectura y depuración del código intermedio generado. El contador por prefijo enCodigoIntermedio.java garantiza unicidad de etiquetas incluso con estructuras anidadas.

Pila separada para break (pilaBreak)

Se utilizó una pila separada (pilaBreak) para el manejo del break, en lugar de usar la pila general de etiquetas (pilaEtiquetas). Esto evita que las etiquetas de estructuras internas (como un if dentro de un while) interfieran con la etiqueta de salida del ciclo cuando se procesa un break.

Generación condicional del código intermedio

El código intermedio solo se genera y muestra si el programa analizado no tiene errores de ningún tipo (léxicos, sintácticos o semánticos). Esta decisión es correcta porque el código intermedio de un programa con errores podría ser incompleto o incorrecto, y mostrarlo podría inducir a confusión.

Temporales para cada operando

Se decidió generar temporales también para literales y variables, no solo para resultados de operaciones. Esto produce un código más detallado y explícito, más cercano al código máquina, donde cada operando se carga explícitamente antes de ser utilizado en una operación.

Switch sin etiqueta default explícita

El último case del switch actúa como el default, eliminando la necesidad de una etiqueta switch_default separada. Esta decisión se tomó porque al intentar mantener una etiqueta

descriptiva `switch_default` independiente, se generaban etiquetas redundantes consecutivas (`switch_case_N`: seguido inmediatamente de `switch_default_1`:) que no aportaban valor semántico y ensuciaban el código intermedio. La solución fue tratar el default como un case más sin condición: cuando ningún case anterior coincide, el flujo cae naturalmente al último `switch_case` gracias a los saltos `if_false` encadenados, actuando exactamente como un default. Esto simplifica el manejo de la pila de etiquetas, elimina la necesidad de crear y gestionar una etiqueta adicional, y produce un código intermedio más limpio sin perder corrección semántica.

Justificación de toma de decisiones: validación semántica y estructuras utilizadas

Para la validación semántica se decidió trabajar directamente dentro de las acciones de CUP. Esto permite revisar cada instrucción en el momento en que el parser la reconoce, sin necesidad de hacer una segunda pasada sobre el código.

La estructura principal usada fue la tabla de símbolos. En ella se registran los identificadores del programa, junto con su tipo de símbolo, tipo de dato, fila, columna y si una variable está inicializada o no. Esto permite validar si una variable existe en el scope actual, si ya fue declarada antes, si se está usando correctamente y si tiene un valor asignado antes de utilizarse.

Los scopes se manejan con una estructura tipo pila. Cada vez que se entra a un nuevo contexto, como una función, el main, un if, un else, un while o un switch, se agrega un nuevo scope. Cuando se sale de ese bloque, el scope se retira. Esto ayuda a separar las variables de cada bloque y evita que se mezclen declaraciones de diferentes partes del programa.

En estructuras como if y while, se valida que la condición sea de tipo bool. Además, dentro de sus bloques se siguen revisando las mismas reglas semánticas del resto del programa, como existencia de variables, compatibilidad de tipos en asignaciones, uso correcto de return y detección de código inalcanzable después de un return.

También se implementó la clase `ErroresSemanticos`, donde se concentran varias validaciones importantes. Ahí se revisan operaciones aritméticas, relacionales y lógicas, además de compatibilidad de tipos. Por ejemplo, se valida que una suma, resta o multiplicación use valores numéricos, que las operaciones lógicas usen valores booleanos y que operadores como la división entera se usen con los tipos permitidos. Esta clase ayudó a reducir código repetido dentro del archivo CUP.

Para los arreglos se valida que, al declararlos, sus dimensiones sean números enteros. Al acceder o modificar una posición del arreglo, también se revisa que el identificador exista, que realmente sea un arreglo y que los índices sean de tipo int. En la modificación, además se permite usar expresiones como índice, siempre que el resultado de esas expresiones sea entero.

Para las funciones, la tabla de símbolos registra el identificador como tipo `FUNCION`, junto con su tipo de retorno. Esto permite validar que una función exista antes de ser llamada y que

no se intente usar una variable como si fuera función. Además, durante el análisis se mantiene el tipo de retorno actual para verificar que las sentencias return coincidan con el tipo declarado de la función. También se valida que una función que no sea empty retorne un valor, y que el main, al ser empty, no retorne valores.

Finalmente, se decidió que el código intermedio solo se genere cuando no existan errores léxicos, sintácticos ni semánticos. Esto evita producir código de tres direcciones a partir de un programa inválido, ya que podría generar instrucciones incompletas o incorrectas.